standard Kubernetes manifests, and it's built natively in the Kubernetes CLI (kubectl), so you don't need to install anything else.

NOTE Other popular options for managing deployment configuration in Kubernetes are ytt from the Carvel suite (https://carvel.dev/ytt) and Helm (https://helm.sh).

This section will show you the key features offered by Kustomize. First you'll see how to *compose* related Kubernetes manifests and handle them as a single unit. Then I'll show you how Kustomize can generate a ConfigMap for you from a property file. Finally, I'll guide you through a series of customizations that we'll apply to the base manifests before deploying workloads in a staging environment. The next chapter will expand on that and cover the production scenario.

Before moving on, make sure you still have your local minikube cluster up and running and that the Polar Bookshop backing services have been deployed correctly. If you don't, run `./create-cluster.sh` from polar-deployment/kubernetes/platform/ development.

NOTE The platform services are exposed only within the cluster. If you want to access any of them from your local machine, you can use the port-forwarding feature you learned about in chapter 7. You can either leverage the GUI provided by Octant or use the CLI (`kubectl port-forward service/polar-postgres 5432:5432`).

Now that we have all the backing services available, let's see how we can manage and configure a Spring Boot application using Kustomize.

14.3.1 Using Kustomize to manage and configure Spring Boot applications

So far, we've been deploying applications to Kubernetes by applying multiple Kubernetes manifests. For example, deploying Catalog Service